

SDLC Using **Agentic AI**

Building an Adaptive AI Software Engineering Organization

Presented By:

Abhishek Raj (M.Tech Intern)

Guide:

M.B. Sai Charan (Scientist C)

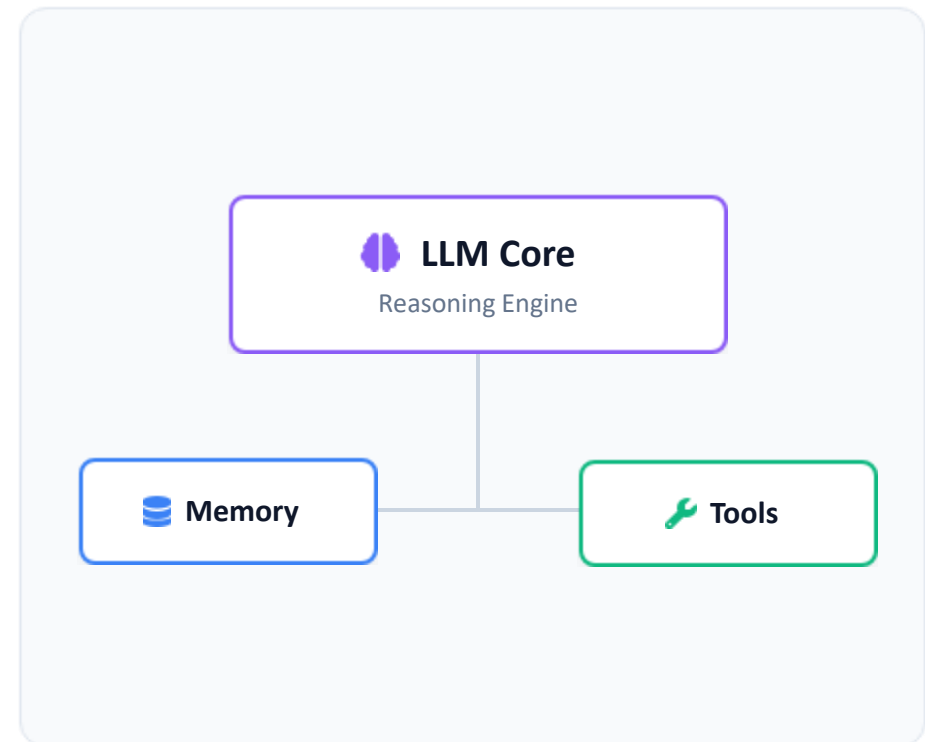
What is an AI Agent?

An **AI Agent** is an autonomous system that uses a Large Language Model (LLM) as its central reasoning engine to actively solve problems.[4]

It observes its environment, breaks down complex goals into actionable steps, and manipulates its surroundings by calling external tools.[1]

The Formula:

$\text{Agent} = \text{LLM} + \text{Memory} + \text{Planning} + \text{Tools}$



Types of AI Agents

Simple Reflex Agents

React solely to current percepts without considering past experiences, following predefined condition–action rules. [4]

- ✓ No memory capacity
- ✓ Rule-based decisions
- ✓ Fast but inflexible

EXAMPLES

Auto doors, Thermostats, Motion lights

Model-Based Agents

Maintain an internal model (state) of the environment, allowing informed decisions despite partial observability. [4]

- ✓ Maintains internal state
- ✓ Handles partial observability
- ✓ Updates knowledge continuously

EXAMPLES

Robot vacuums, Warehouse robots

Goal-Based Agents

Choose actions by evaluating future states and planning a sequence of steps to achieve a specific objective. [4]

- ✓ Goal-oriented approach
- ✓ Planning & search capabilities
- ✓ Evaluates possible actions

EXAMPLES

GPS navigation, Logistics planners

Utility-Based Agents

Evaluate multiple possible actions using a utility function, selecting the option that maximizes overall performance and handles trade-offs efficiently. [4]

- ✓ Decision optimization
- ✓ Handles complex trade-offs
- ✓ Maximizes expected reward/utility

EXAMPLES

Algorithmic trading, Dynamic pricing, Autonomous driving

Learning Agents

Improve behavior autonomously over time by continuously learning from data, user feedback, and environment interactions. [4]

- ✓ Self-improving over time
- ✓ Learns directly from experience
- ✓ Adapts to changing environments

EXAMPLES

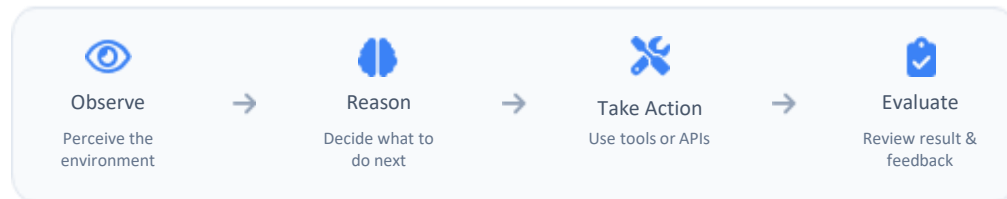
AlphaGo, Netflix Recommendations, Adaptive spam filters

AI Agent Architectures

Different patterns for designing agents based on how they reason, act, and collaborate.

1. ReAct (Reason & Act)

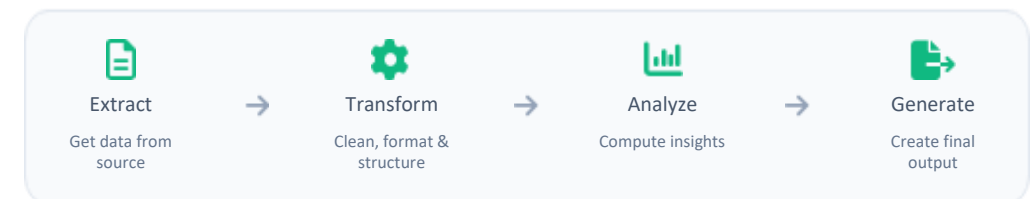
A single agent follows an iterative loop: observe the environment, reason about what to do next, act using tools, evaluate the result, and repeat until the task is complete.



Example Use Cases: Q&A, Code generation & debugging

2. Sequential Workflow

A pipeline where tasks are executed in a fixed, linear order. The output of one step becomes the input to the next.



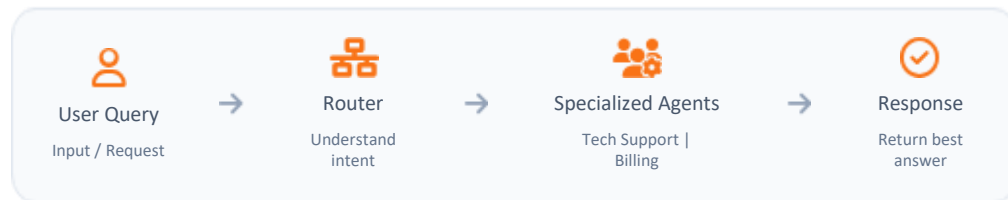
Example Use Cases: Document processing pipeline, Report generation

AI Agent Architectures

Different patterns for designing agents based on how they reason, act, and collaborate.

3. Routing Architecture

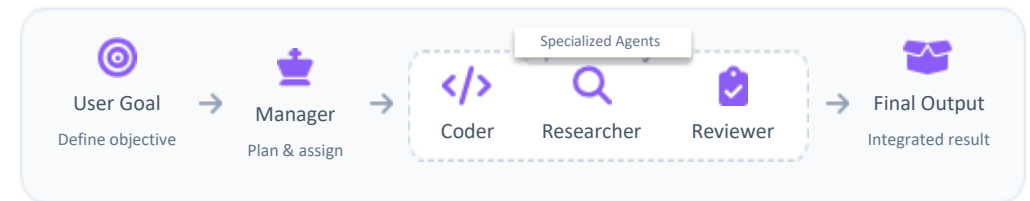
A supervisor (router) interprets the intent of the prompt and routes it to the most suitable specialized worker agent.



Example Use Cases: Customer support automation

4. Multi-Agent Swarm

Multiple agents with different roles collaborate, communicate, and debate to solve complex problems under a coordinating agent.



Example Use Cases: Software development teams, Threat analysis & support

The Traditional SDLC

The Traditional Software Development Life Cycle (SDLC) is the structured framework that engineering teams follow to plan, design, build, test, and deploy software applications from initial concept.



This Manual SDLC execution creates severe friction—engineers waste most of their time on administrative handoffs, environment setups, regression testing, and ticketing overhead instead of writing core logic.[5]

So, Our Aim is to automate the Traditional SDLC with Agentic AI, which will replace these static bottlenecks with continuous, self-healing pipelines that autonomously execute tests, patch build failures, flag vulnerabilities, and manage releases. This **eliminates human error, compresses cycle times from months to hours, and frees development teams to focus on high-value architecture and innovation.**

Beyond Code Generation: Agentic AI

Agentic AI consists of autonomous AI agents capable of planning, reasoning, using tools, maintaining memory, and collaborating to achieve complex goals.

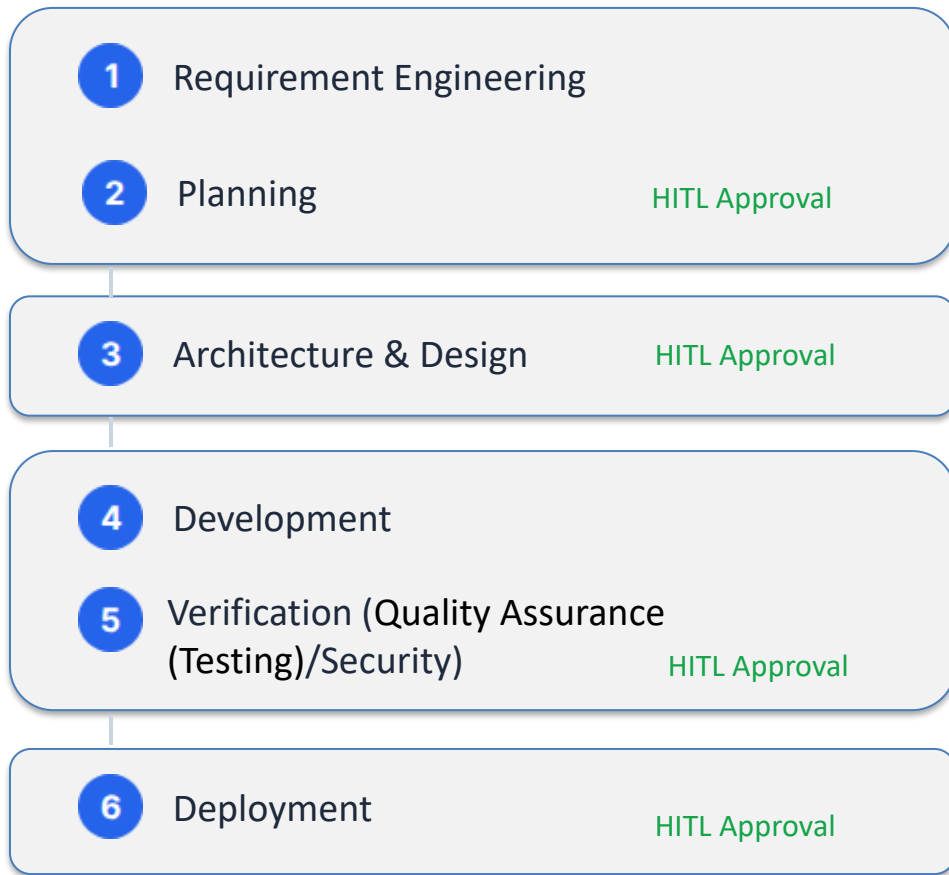
Capability	Standard Chatbot	Agentic AI
Execution	Passive (Answers queries)	Autonomous (Acts & Uses Tools)
Workflow	Single-step generation	Multi-step reasoning & execution
Context	No Memory (Session bound)	Persistent Memory (Context aware)
Strategy	No Planning	Goal Planning & Reflection

Agentic Frameworks

Framework	Execution Model	Core Strengths	Key Limitations	Ideal Use Cases	Offline / Local Capability
LangChain [6]	Chained Prompting & Tools	Rich toolset, extensive RAG ecosystem, vast API integrations	Workflow complexity at scale, abstractions can hide errors	General LLM applications, retrieval-augmented generation	Excellent: Natively connects to local LLMs (Ollama, Llama.cpp) and local vector stores (Chroma).
LangGraph [7]	Stateful Graph Orchestration	Cycles and loops, robust state persistence, time-travel debugging	Steeper learning curve, excessive setup for simple tasks	Production-grade multi-agent systems, human-in-the-loop control	Excellent: Inherits LangChain's local capabilities; operates entirely offline with self-hosted models.
CrewAI [8]	Role-based Team Collaboration	Easy setup, intuitive agent personas, automatic task delegation	Limited dynamic branching and complex workflow flow control	Autonomous research pipelines, content creation, structured teams	Strong: Explicitly supports local execution by binding agents to local endpoints via Ollama or LiteLLM.
OpenAI Agents SDK [9]	Lightweight Handoffs & Guardrails	Minimal abstractions, built-in tracing, standardized routing primitives	Lacks built-in persistent long-term memory or native graph structures	Rapid deployment of production agents, voice pipelines, clear delegation	Very Good: Provider-agnostic design allows it to run offline by routing to local models via the Chat Completions API.

Proposed Architecture: Modular Pipeline

A modular pipeline with integrated **Human-in-the-Loop (HITL)** governance gates to ensure enterprise safety.

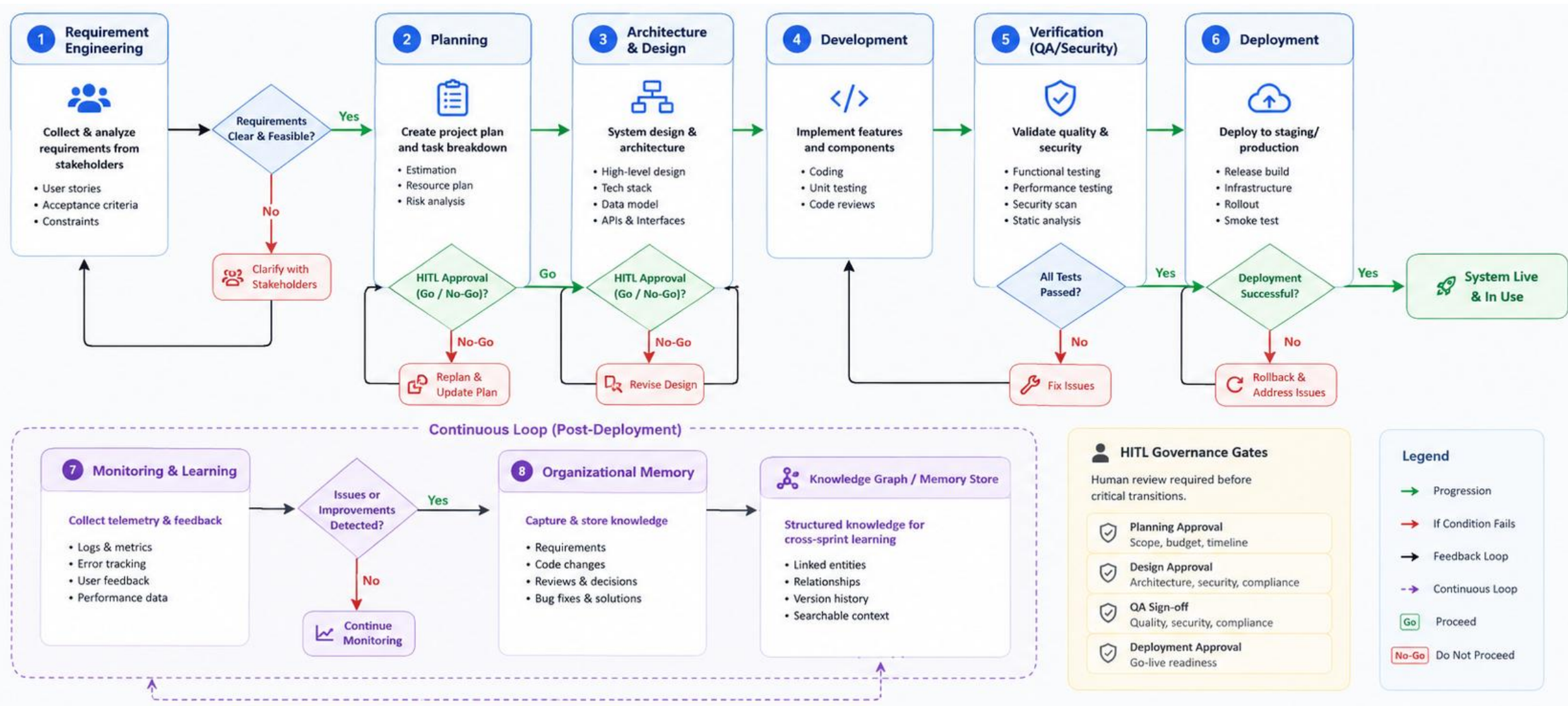


Continuous Loop

7. Monitoring & Learning: Consumes production logs, metrics, and user feedback to detect errors and optimize prompts.

8. Organizational Memory: Captures all requirements, code, reviews, and bug fixes into a Knowledge Graph for cross-sprint learning.

Proposed Architecture: Modular Pipeline



Identifying the Gap

While highly capable, existing autonomous systems index heavily on code generation, lacking end-to-end SDLC orchestration and organizational memory.

System	Requirements	Design	Coding	Testing	Deployment
Devin / OpenHands [13][14]	Partial	Partial	Yes	Yes	Partial
MetaGPT / ChatDev [10][11]	Yes	Yes	Yes	Partial	No
SWE-Agent [12]	No	No	Yes	Yes	No
Proposed System	Yes	Yes	Yes	Yes	Yes

Core Innovation: Adaptive Team Generation

Traditional: Fixed Agents

Standard agentic systems rely on rigid, hardcoded workflows (e.g., always Planner → Developer → Tester).

- Inefficient for small tasks (over-engineered).
- Lacks specialized roles (Security, DevOps) for complex enterprise apps.

Proposed: Dynamic Generation

The Orchestrator analyzes project complexity, risk, and size before execution to spawn agents on-the-fly.

- **Small Script:** Spawns Planner + Developer.
- **Enterprise App:** Spawns Architect, Backend, Frontend, Security, QA, DevOps.
- Agents are retired when tasks complete, saving compute.

Core Innovation: Hierarchical Software Knowledge Graph (HSKG)

Beyond Traditional Memory:
A Context-Aware Project Memory

The Problem: Current AI agents retrieve entire repositories or large documents, resulting in high token consumption and irrelevant context.

Our Solution: Our proposed HSKG stores the software project as a hierarchical graph, enabling agents to retrieve only the smallest relevant subgraph required for a task.



Project Graph

Project

- └ Authentication
 Login, Register
- └ Inventory
 Product, Stock
- └ Invoice
 Create, Payment

Purpose

- Represents the complete SDLC structure
- Maintains hierarchy between project components
- Enables modular navigation



Context-Aware

Instead of:
Entire Repository → LLM

Agent retrieves:
Login Module → UI → Backend Script & Variables
→ API → User DB

Benefits

- Minimal context
- Lower token usage
- Faster reasoning
- Better accuracy
- Reduced hallucination

Local Offline Execution Strategy

Deploying via offline models ensures data privacy. Model selection is based on VRAM footprint, Training data freshness, Suitable for Agentic Operations.

Model (4-bit Quant)	Recommended Hardware (VRAM)	Knowledge Cutoff	Agent Capability	Best Use Case
Gemma 4 4B	Laptop / iGPU (~5 GB)	Early 2026	★ ★ ★	Basic assistants, RAG, lightweight offline agents
Gemma 4 12B	RTX 4070 (8–10 GB)	Mid 2026	★ ★ ★ ★	General-purpose AI agents with good reasoning & tool use
Mistral Small 3.2 (24B)	RTX 4090 (16 GB)	Mid 2025	★ ★ ★ ★ ★	Enterprise agents, multi-tool workflows, planning
Qwen3-Coder 32B	Workstation / Mac M-Series (20–22 GB)	Mid 2025	★ ★ ★ ★ ★	Coding agents, debugging, software engineering
Llama 3.3 70B	Multi-GPU / High-End Server (48+ GB)	Late 2023*	★ ★ ★ ★ ★	Research, complex planning, multi-agent orchestration

Source: Compiled from official model documentation and validated using publicly available implementation benchmarks and deployment experiences.

Conclusion

We are proposing an **Autonomous Software Development Lifecycle (A-SDLC)** framework that transforms traditional software engineering into an **AI-Orchestrated Software Engineering Organization**.

Instead of isolated coding assistants, the system coordinates multiple specialized AI agents across the complete SDLC—from **requirements engineering** to **continuous monitoring and learning**.

Expected Outcomes

- ✓ Reduced context size and token consumption
- ✓ Faster and more accurate software development
- ✓ Improved traceability across the SDLC
- ✓ Persistent engineering knowledge

References

Research Papers & Background

- [1] He, J., Treude, C., & Lo, D. (2025). *LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision...* ACM TOSEM.
- [2] Apostolou, S. A., et al. (2026). *Assistance to Autonomy: A Systematic Literature Review of Agentic AI across the SDLC*. arXiv.
- [3] *Agentic AI in the Software Development Lifecycle: Architecture, Empirical Evidence...* (2026). arXiv.
- [4] Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach (4th Edition)*.
- [5] Sommerville, I. *Software Engineering (10th Edition)*.

Frameworks & Systems

- [6] *LangChain Documentation*.
- [7] *LangGraph Documentation*.
- [8] *CrewAI Documentation*.
- [9] *OpenAI Agents SDK Documentation*.
- [10] Hong, S., et al. (2024). *MetaGPT: Meta Programming for Multi-Agent Collaborative Framework*.
- [11] Qian, C., et al. (2024). *ChatDev: Communicative Agents for Software Development*.
- [12] Yang, J., et al. (2024). *SWE-Agent: Agent-Computer Interfaces Enable Automated SE*.
- [13] *OpenHands Project*.
- [14] Cognition AI – *Devin: AI Software Engineer*.

Thank You